

Sorting

Concert Tickets

```
#include

using namespace std;

int N, M, h, t;
multiset prices;

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < N; i++){
        scanf("%d", &h);
        prices.insert(-h);
    }
    for(int i = 0; i < M; i++){
        scanf("%d", &t);
        if(prices.lower_bound(-t) == prices.end())
            printf("-1\n");
        else {
            printf("%d\n", -(*prices.lower_bound(-t)));
            prices.erase(prices.lower_bound(-t));
        }
    }
}
```

Restaurant Customers

```
#include

using namespace std;
const int maxN = 4e5;

int N, a, b, cur, ans;
struct event {int time, type;} events[maxN];

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d %d", &a, &b);
        events[2*i] = {a, 1};
        events[2*i+1] = {b, -1};
    }
    sort(events, events+2*N, [](event A, event B){
        return A.time == B.time ? A.type < B.type : A.time < B.time;
    });

    for(int i = 0; i < 2*N; i++){
        cur += events[i].type;
        ans = max(ans, cur);
    }
}
```

```
printf("%d\n", ans);  
}
```

Movie Festival

```
#include  
  
using namespace std;  
typedef pair pii;  
const int maxN = 2e5+1;  
  
int N, a[maxN], b[maxN], dp[2*maxN];  
struct Movie {int t, id, type;} movies[2*maxN];  
map mp;  
  
int main(){  
    scanf("%d", &N);  
    for(int i = 0; i < N; i++){  
        scanf("%d %d", &a[i], &b[i]);  
        movies[2*i] = {a[i], i, 0};  
        movies[2*i+1] = {b[i], i, 1};  
    }  
  
    sort(movies, movies+2*N, [](Movie A, Movie B){  
        return A.t == B.t ? A.id < B.id : A.t < B.t;  
    });  
  
    for(int i = 0; i < 2*N; i++){  
        mp[movies[i].t] = i;  
    }  
  
    for(int i = 0; i < 2*N; i++){  
        if(movies[i].type == 0) dp[i] = dp[i-1];  
        else dp[i] = max(dp[i-1], dp[mp[a[movies[i].id]]]+1);  
    }  
    printf("%d\n", dp[2*N-1]);  
}
```

Sum of Two Values

```
#include  
  
using namespace std;  
typedef pair pii;  
const int maxN = 2e5+1;  
  
int N, x, a;  
pii p[maxN];  
  
int find(int val){  
    int l = 1, r = N;  
    while(l <= r){  
        int m = l+(r-l)/2;
```

```

        if(p[m].first == val)        return p[m].second;
        else if(p[m].first < val)    l = m+1;
        else                        r = m-1;
    }
    return 0;
}

int main(){
    scanf("%d %d", &N, &x);
    for(int i = 1; i <= N; i++){
        scanf("%d", &a);
        p[i] = {a, i};
    }
    sort(p+1, p+N+1);

    for(int i = 1; i <= N; i++){
        int other = find(x-p[i].first);
        if(other != 0 && other != p[i].second){
            printf("%d %d\n", p[i].second, other);
            return 0;
        }
    }
    printf("IMPOSSIBLE\n");
}

```

Maximum Subarray Sum

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;
const ll INF = 0x3f3f3f3f3f3f3f3f;

int N;
ll maxSum, curSum, x[maxN];

int main(){
    scanf("%d", &N);
    maxSum = -INF;
    for(int i = 0; i < N; i++){
        scanf("%lld", &x[i]);
        maxSum = max(maxSum, x[i]);
    }

    for(int i = 0; i < N; i++){
        curSum += x[i];
        maxSum = max(maxSum, curSum);
        if(curSum < 0) curSum = 0;
    }
    printf("%lld\n", maxSum);
}

```

Stick Lengths

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N, p[maxN];
ll median, sum;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        scanf("%d", &p[i]);
    sort(p, p+N);

    median = p[(N-1)/2];
    for(int i = 0; i < N; i++)
        sum += abs(p[i]-median);
    printf("%lld\n", sum);
}
```

Missing Coin Sum

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N, x[maxN];
ll res;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        scanf("%d", &x[i]);
    sort(x, x+N);

    res = 1;
    for(int i = 0; i < N && x[i] <= res; i++)
        res += x[i];
    printf("%lld\n", res);
}
```

Collecting Numbers

```
#include
```

```
using namespace std;
const int maxN = 2e5+1;

int N, x, cnt, pos[maxN];

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf("%d", &x);
        pos[x] = i;
    }

    cnt = 1;
    for(int i = 2; i <= N; i++)
        if(pos[i-1] > pos[i])
            cnt++;
    printf("%d\n", cnt);
}
```

Collecting Numbers II

```
#include

using namespace std;
const int maxN = 2e5+5;

int N, M, cnt, x[maxN], pos[maxN];

void update(int a, int b){
    if(pos[x[a]-1] <= pos[x[a]] && b < pos[x[a]-1]) cnt++;
    if(pos[x[a]-1] > pos[x[a]] && b >= pos[x[a]-1]) cnt--;
    if(pos[x[a]+1] >= pos[x[a]] && b > pos[x[a]+1]) cnt++;
    if(pos[x[a]+1] < pos[x[a]] && b <= pos[x[a]+1]) cnt--;
    pos[x[a]] = b;

    if(pos[x[b]-1] <= pos[x[b]] && a < pos[x[b]-1]) cnt++;
    if(pos[x[b]-1] > pos[x[b]] && a >= pos[x[b]-1]) cnt--;
    if(pos[x[b]+1] >= pos[x[b]] && a > pos[x[b]+1]) cnt++;
    if(pos[x[b]+1] < pos[x[b]] && a <= pos[x[b]+1]) cnt--;
    pos[x[b]] = a;

    swap(x[a], x[b]);
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 1; i <= N; i++){
        scanf("%d", &x[i]);
        pos[x[i]] = i;
    }
    pos[N+1] = N+1;

    cnt = 1;
    for(int i = 1, ptr = 0; i <= N; i++){
        if(ptr > pos[i])
            cnt++;
        ptr = pos[i];
    }

    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
```

```
        update(a, b);
        printf("%d\n", cnt);
    }
}
```

Playlist

```
#include

using namespace std;

int N, k, l, ans;
map mp;

int main(){
    scanf("%d", &N);

    l = 1;
    for(int r = 1; r <= N; r++){
        scanf("%d", &k);
        if(mp[k]){
            ans = max(ans, r-1);
            l = max(l, mp[k]+1);
            mp[k] = r;
        } else {
            ans = max(ans, r-1+1);
            mp[k] = r;
        }
    }

    ans = max(N-l+1, ans);
    printf("%d\n", ans);
}
```

Towers

```
#include

using namespace std;

int N, k;
multiset S;
multiset::iterator it;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d", &k);
        it = S.upper_bound(k);
        if(it != S.end())
            S.erase(it);
        S.insert(k);
    }
}
```

```

    }
    printf("%d\n", (int) S.size());
}

```

Traffic Lights

```

#include

using namespace std;
const int maxN = 2e5;

set S;
set::iterator f, c, b;
int N, X, a, best, ans[maxN];
struct light {int pos, id;} x[maxN+2];

int main(){
    scanf("%d %d", &X, &N);
    x[0] = {0, 0}; x[N+1] = {X, 1};
    S.insert(0); S.insert(X);
    for(int i = 0; i < N; i++){
        scanf("%d", &a);
        x[i+1] = {a, i+2};
        S.insert(a);
    }

    sort(x, x+maxN+2, [](light A, light B){
        return A.pos < B.pos;
    });
    for(int i = 1; i <= maxN+1; i++)
        best = max(best, x[i].pos - x[i-1].pos);

    sort(x, x+maxN+2, [](light A, light B){
        return A.id > B.id;
    });
    for(int i = 0; i < N; i++){
        ans[N-i-1] = best;
        c = S.find(x[i].pos);
        f = next(c, 1);
        b = next(c, -1);
        best = max(best, *f - *b);
        S.erase(c);
    }

    for(int i = 0; i < N; i++)
        printf("%d%c", ans[i], (" \n")[i==N]);
}

```

Distinct Values Subarrays

```

#include

```

```

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, r, x[maxN];
ll ans;
set S;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d", &x[i]);
    }

    for(int l = 0; l < N; l++){
        while(r < N && S.find(x[r]) == S.end()){
            S.insert(x[r]);
            r++;
        }
        ans += r - l;
        S.erase(x[l]);
    }
    printf("%lld\n", ans);
}

```

Distinct Values Subsequences

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;
const ll MOD = 1e9+7;

int N, x[maxN];
map freq;

ll mod_inv(ll a){
    ll res = 1;
    ll b = MOD-2;
    while(b > 0){
        if(b&1)
            res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        scanf("%d", &x[i]);

    ll ans = 0;
    ll prod = 1;
    for(int i = 0; i < N; i++){
        ll seqs = (prod * mod_inv(freq[x[i]]+1)) % MOD;
        ans = (ans + seqs) % MOD;

        freq[x[i]]++;
    }
}

```

```

        prod = (prod * (freq[x[i]]+1) ) % MOD;
        prod = (prod * mod_inv(freq[x[i]])) % MOD;
    }
    printf("%lld\n", ans);
}

```

Josephus Problem I

```

#include
#include
#include

using namespace std;
using namespace __gnu_pbds;

int N, idx;
tree, rb_tree_tag,
tree_order_statistics_node_update> T;

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++)
        T.insert(i);

    idx = 1;
    while(T.size()){
        idx %= T.size();
        int x = *T.find_by_order(idx);
        T.erase(x);
        printf("%d%c", x, (" \n")[T.size() == 0]);
        idx++;
    }
}

```

Josephus Problem II

```

#include
#include
#include

using namespace std;
using namespace __gnu_pbds;

int N, K, idx;
tree, rb_tree_tag,
tree_order_statistics_node_update> T;

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 1; i <= N; i++)
        T.insert(i);
}

```

```

    idx = K;
    while(T.size()){
        idx %= T.size();
        int x = *T.find_by_order(idx);
        T.erase(x);
        printf("%d%c", x, (" \n")[T.size() == 0]);
        idx += K;
    }
}

```

Nested Ranges Check

```

#include

using namespace std;
typedef array triple;
const int maxN = 2e5+1;
const int SIZE = 2*maxN;

int N, ans[2][maxN], ds[SIZE];
triple intervals[maxN];
set unique_vals;
map mp;

void update(int idx, int val){
    for(int i = idx; i < SIZE; i += -i&i)    ds[i] += val;
}

int query(int idx){
    int cnt = 0;
    for(int i = idx; i > 0; i -= -i&i)
        cnt += ds[i];
    return cnt;
}

int main(){
    scanf("%d", &N);
    for(int i = 0, x, y; i < N; i++){
        scanf("%d %d", &x, &y);
        intervals[i] = {x, y, i};
        unique_vals.insert(x);
        unique_vals.insert(y);
    }
    sort(intervals, intervals+N, [](triple A, triple B){
        return (A[0] == B[0] ? A[1] > B[1] : A[0] < B[0]);
    });

    int val_id = 1;
    for(int x : unique_vals)
        mp[x] = val_id++;

    for(int i = N-1; i >= 0; i--){
        int y = mp[intervals[i][1]];
        int id = intervals[i][2];

        ans[0][id] = query(y);
        update(y, 1);
    }

    fill(ds, ds+SIZE, 0);
    for(int i = 0; i < N; i++){

```

```

    int y = mp[intervals[i][1]];
    int id = intervals[i][2];

    ans[1][id] = i - query(y - 1);
    update(y, 1);
}

for(int i = 0; i < 2; i++)
    for(int j = 0; j < N; j++)
        printf("%d%c", (ans[i][j] ? 1 : 0), (" \n")[j==N-1]);
}

```

Nested Ranges Count

```

#include

using namespace std;
typedef array triple;
const int maxN = 2e5+1;
const int SIZE = 2*maxN;

int N, ans[2][maxN], ds[SIZE];
triple intervals[maxN];
set unique_vals;
map mp;

void update(int idx, int val){
    for(int i = idx; i < SIZE; i += -i&i)    ds[i] += val;
}

int query(int idx){
    int cnt = 0;
    for(int i = idx; i > 0; i -= -i&i)
        cnt += ds[i];
    return cnt;
}

int main(){
    scanf("%d", &N);
    for(int i = 0, x, y; i < N; i++){
        scanf("%d %d", &x, &y);
        intervals[i] = {x, y, i};
        unique_vals.insert(x);
        unique_vals.insert(y);
    }
    sort(intervals, intervals+N, [](triple A, triple B){
        return (A[0] == B[0] ? A[1] > B[1] : A[0] < B[0]);
    });

    int val_id = 1;
    for(int x : unique_vals)
        mp[x] = val_id++;

    for(int i = N-1; i >= 0; i--){
        int y = mp[intervals[i][1]];
        int id = intervals[i][2];

        ans[0][id] = query(y);
        update(y, 1);
    }
}

```

```

fill(ds, ds+SIZE, 0);
for(int i = 0; i < N; i++){
    int y = mp[intervals[i][1]];
    int id = intervals[i][2];

    ans[1][id] = i-query(y-1);
    update(y, 1);
}

for(int i = 0; i < 2; i++)
    for(int j = 0; j < N; j++)
        printf("%d%c", ans[i][j], (" \n")[j==N-1]);
}

```

Room Allocation

```

#include

using namespace std;
const int maxN = 2e5;

set S;
int N, a, b, rooms, ans[maxN+1];
struct query {int type, time, id;} queries[2*maxN];

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        S.insert(i+1);
        scanf("%d %d", &a, &b);
        queries[2*i] = {0, a, i+1};
        queries[2*i+1] = {1, b, i+1};
    }

    sort(queries, queries+2*N, [](query A, query B){
        if(A.id == B.id) return A.type < B.type;
        return A.time == B.time ? A.type < B.type : A.time < B.time;
    });

    for(int i = 0; i < 2*N; i++){
        if(queries[i].type == 0){
            ans[queries[i].id] = *S.begin();
            S.erase(S.begin());
        } else if(queries[i].type == 1){
            S.insert(ans[queries[i].id]);
        }
    }

    for(int i = 1; i <= N; i++)
        rooms = max(rooms, ans[i]);

    printf("%d\n", rooms);
    for(int i = 1; i <= N; i++)
        printf("%d%c", ans[i], (" \n")[i==N]);
}

```

Factory Machines

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N;
ll T, cnt, k[maxN];

bool check(ll t){
    cnt = 0;
    for(int i = 0; i < N; i++){
        cnt += t/k[i];
        if(cnt >= T)
            return true;
    }
    return false;
}

int main(){
    scanf("%d %lld", &N, &T);
    for(int i = 0; i < N; i++)
        scanf("%lld", &k[i]);

    ll lo = 0, hi = 1e18;
    while(lo <= hi){
        ll mid = lo + (hi-lo)/2;
        if(check(mid)) hi = mid-1;
        else lo = mid+1;
    }
    printf("%lld\n", lo);
}
```

Tasks and Deadlines

```
#include

using namespace std;
const int maxN = 2e5;

int N, a, d;
long long timer, reward;
struct task {int a, d;} tasks[maxN];

int main (){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d %d", &a, &d);
        tasks[i] = {a, d};
    }

    sort(tasks, tasks+N, [](task x, task y){
        return x.a == y.a ? x.d < y.d : x.a < y.a;
    });

    for(int i = 0; i < N; i++){
        timer += tasks[i].a;
    }
}
```

```
        reward += (tasks[i].d - timer);
    }
    printf("%lld\n", reward);
}
```

Reading Books

```
#include

using namespace std;
typedef long long ll;

int N;
ll t, mx, sum;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%lld", &t);
        mx = max(mx, t);
        sum += t;
    }
    printf("%lld\n", max(sum, 2*mx));
}
```

Sum of Three Values

```
#include

using namespace std;
typedef pair pii;
const int maxN = 5000;

int N, X, v, r;
pii a[maxN];

int main(){
    scanf("%d %d", &N, &X);
    for(int i = 0; i < N; i++){
        scanf("%d", &v);
        a[i] = {v, i+1};
    }

    sort(a, a+N);
    for(int i = 0; i < N; i++){
        v = X - a[i].first;
        r = N-1;
        for(int l = i+1; l < r; l++){
            while(l+1 < r && a[l].first + a[r].first > v)    r--;
            if(a[l].first + a[r].first == v){
                printf("%d %d %d\n", a[i].second, a[l].second, a[r].second);
                return 0;
            }
        }
    }
}
```

```

    }
}
}
printf("IMPOSSIBLE\n");
}

```

Sum of Four Values

```

#include

using namespace std;
typedef long long ll;
typedef pair pii;
const int maxN = 1001;

int N;
ll X, a[maxN];
unordered_map> mp;

int main(){
    scanf("%d %lld", &N, &X);
    mp.reserve(maxN*maxN);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &a[i]);
        for(int j = 1; j < i; j++){
            ll psum = a[i] + a[j];
            if(psum >= X) continue;
            if(mp.find(X-psum) != mp.end()){
                for(pii P : mp[X-psum]){
                    if(P.first != j && P.second != i && P.first != i && P.second != j){
                        printf("%d %d %d %d\n", P.first, P.second, j, i);
                        return 0;
                    }
                }
            } else mp[psum].push_back({j, i});
        }
    }
    printf("IMPOSSIBLE\n");
}

```

Nearest Smaller Values

```

#include

using namespace std;
const int maxN = 2e5+1;

int N, x[maxN], ds[maxN];

int main(){
    scanf("%d ", &N);
    for(int i = 1; i <= N; i++){

```

```

scanf("%d", &x[i]);
int k = i-1;
while(x[k] >= x[i])
    k = ds[k];
ds[i] = k;
printf("%d%c", ds[i], (" \n")[i==N]);
}
}

```

Subarray Sums I

```

#include

using namespace std;
const int maxN = 2e5;

int N, x, l, sum, cnt, a[maxN+1];

int main(){
    scanf("%d %d", &N, &x);
    for(int r = 1; r <= N; r++){
        scanf("%d", &a[r]);
        sum += a[r];
        while(sum > x)
            sum -= a[l++];
        if(sum == x)    cnt++;
    }
    printf("%d\n", cnt);
}

```

Subarray Sums II

```

#include

using namespace std;
typedef long long ll;

int N;
ll X, a, cnt, pre;
map freq;

int main(){
    scanf("%d %lld", &N, &X);
    freq[0] = 1;
    for(int i = 0; i < N; i++){
        scanf("%lld", &a);
        pre += a;
        cnt += freq[pre-X];
        freq[pre]++;
    }
}

```

```
    printf("%lld\n", cnt);  
}
```

Subarray Divisibility

```
#include  
  
using namespace std;  
typedef long long ll;  
const int maxN = 2e5;  
  
int N, pre, a, freq[maxN];  
ll cnt;  
  
int mod(int x){  
    return (x % N + N) % N;  
}  
  
int main(){  
    scanf("%d", &N);  
    freq[0] = 1;  
    for(int i = 0; i < N; i++){  
        scanf("%d", &a);  
        pre = mod(pre+a);  
        cnt += freq[pre];  
        freq[pre]++;  
    }  
    printf("%lld\n", cnt);  
}
```

Distinct Values Subarrays II

```
#include  
  
using namespace std;  
typedef long long ll;  
const int maxN = 2e5+5;  
  
int N, K, x[maxN];  
map freq;  
ll ans;  
  
int main(){  
    scanf("%d %d", &N, &K);  
    for(int i = 1; i <= N; i++)  
        scanf("%d", &x[i]);  
  
    int unique = 0;  
    int r = 0;  
    for(int l = 1; l <= N; l++){  
        while(r < N && (freq[x[r+1]] >= 1 || unique < K)){  
            r++;  
        }  
        ans += (r - l + 1);  
        freq[x[l]]++;  
        unique++;  
    }  
    printf("%lld\n", ans);  
}
```

```
        freq[x[r]]++;
        if(freq[x[r]] == 1)
            unique++;
    }

    ans += (r-l+1);

    freq[x[l]]--;
    if(!freq[x[l]])
        unique--;
}

printf("%lld\n", ans);
}
```

Array Division

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5;

int N, K, a[maxN];
ll lo = 0, hi = 1e18;

bool check(ll X){
    int k = 1;
    ll sum = 0;
    for(int i = 0; i < N; i++){
        sum += a[i];
        if(sum > X){
            k++;
            sum = a[i];
        }
    }
    return k <= K;
}

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < N; i++){
        scanf("%d", &a[i]);
        lo = max(lo, (ll) a[i]);
    }

    while(lo <= hi){
        ll mid = lo + (hi-lo)/2;
        if(check(mid)) hi = mid-1;
        else lo = mid+1;
    }
    printf("%lld\n", lo);
}
```

Movie Festival II

```
#include

using namespace std;
const int maxN = 2e5, SIZE = 5e6;
const int INF = 0x3f3f3f3f;

int N, K, ans, A[maxN], B[maxN], lo[SIZE], hi[SIZE], d[SIZE], mx[SIZE];
struct Movie { int t, id, type; } movies[2*maxN];
map mp;

void push(int i){
    if(d[i]){
        d[2*i] += d[i];
        d[2*i+1] += d[i];
        d[i] = 0;
    }
}

void pull(int i){
    mx[i] = max(mx[2*i]+d[2*i], mx[2*i+1]+d[2*i+1]);
}

void init(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r) return;
    int m = l+(r-l)/2;
    init(2*i, l, m);
    init(2*i+1, m+1, r);
    pull(i);
}

void increment(int i, int l, int r){
    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        d[i]++; return;
    }
    push(i);
    increment(2*i, l, r);
    increment(2*i+1, l, r);
    pull(i);
}

int maximum(int i, int l, int r){
    if(l > hi[i] || r < lo[i]) return -INF;
    if(l <= lo[i] && hi[i] <= r) return mx[i]+d[i];

    push(i);
    int lmax = maximum(2*i, l, r);
    int rmax = maximum(2*i+1, l, r);
    pull(i);

    return max(lmax, rmax);
}

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < N; i++){
        scanf("%d %d", &A[i], &B[i]);
        movies[2*i] = {A[i], i, 0};
        movies[2*i+1] = {B[i], i, 1};
    }
    sort(movies, movies+2*N, [](Movie a, Movie b){
        return a.t == b.t ? A[a.id] > A[b.id] : a.t < b.t;
    });

    for(int i = 0; i < 2*N; i++)
```

```

        mp[movies[i].t] = 2*i+1;

    init(1, 1, 4*N);
    for(int i = 0; i < 2*N; i++){
        if(movies[i].type == 1){
            int id = movies[i].id;
            int a = mp[A[id]];
            int b = mp[B[id]];

            if(maximum(1, a, b) < K){
                increment(1, a, b-1);
                ans++;
            }
        }
    }
    printf("%d\n", ans);
}

```

Maximum Subarray Sum II

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, A, B;
ll pre[maxN];
multiset S;

int main(){
    scanf("%d %d %d", &N, &A, &B);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &pre[i]);
        pre[i] += pre[i-1];
    }

    for(int i = A; i <= B; i++)
        S.insert(pre[i]);

    ll best = *S.rbegin();
    for(int i = 1; i+A <= N; i++){
        S.erase(pre[i+A-1]);
        S.insert(pre[min(i+B, N)]);
        best = max(best, *S.rbegin()-pre[i]);
    }

    printf("%lld\n", best);
}

```